

Hinweise

Das ist ein bewertetes Übungsblatt, welches für das Bestehen des Modules „Einführung in die Algorithmik“ relevant ist. Bitte beachten Sie folgende Hinweise:

- Erstellen Sie für die Abgabe des Blattes eine lesbare PDF Datei. Erstellen Sie für die Python Programme nur genau eine Datei, in welcher Sie mit Kommentaren kennzeichnen, um welche Aufgabenlösungen es sich handelt. Dies kann beispielsweise wie folgt aussehen:

```
#####
# Übung 1:
#####
# Übung 1-1
def quantumfoo():
```

- Geben Sie dieses Blatt nur in Gruppen von mindestens zwei und maximal vier Personen ab.
- Um Personen für Ihre Übungsgruppe zu finden nutzen Sie bitte die vorgesehenen Foren in StudOn oder ihr Tutorium.
- Benennen Sie die Dateien, welche Sie abgeben wollen nach folgendem Schema, wobei < und > nicht enthalten sein sollen:
 - Schema: hw<ÜbungsNr.>.<Vornamen>.<Nachname>.<Vornamen>.<Nachname>.*
 - Beispiel 3er Gruppe für Übung 2: hw2_Hongjun_Wu_Tanja_Lange_Jonathan_Katz.pdf
- Nutzen Sie zur Abgabe die dafür vorgesehene Umgebung in StudOn.
- Achten Sie auf eine saubere und lesbare Abgabe. Wir empfehlen die Verwendung eines auf T_EX basierenden Textsatzsystems wie beispielsweise L^AT_EX. Den korrigierenden Personen ist es nach eigenem Ermessen gestattet, nicht lesbare Lösungen mit 0 Punkten zu bewerten.
- Wenn Sie in Ihrer PDF-Agabe auf Ihre Programme oder Teile davon verweisen müssen, empfehlen wir die Verwendung des L^AT_EX-Pakets *minted*.
- **Bewertung:** Insgesamt können 50 Punkte pro bewertetem Übungsblatt erzielt werden. Es wird 4 bewertete Blätter geben. Sie brauchen insgesamt mindestens 100 Punkte um die Übung zu bestehen.
- Aus didaktischen Gründen sind die nutzbaren Funktionalitäten von Python eingeschränkt. Die zugelassenen Funktionalitäten finden Sie in den Einführungsfolien zu Python oder als expliziten Hinweis bei den Aufgabenstellungen.

Aufgabe:	1.1	1.2	2.1	2.2	Summe:
Punkte:	21	12	13	4	50
Ergebnis:					

1 Optimierter Lexiographischer Quicksort

9 7 8 3 1 2 5

Step 1: Wähle ein Pivot Element (Strategie des letzten Elementes)

9 7 8 3 1 2 5

Step 2: Kleinere Werte gehen nach links, gleiche oder größere Werte gehen nach rechts

3 1 2 5 9 7 8

Step 3: Wiederholen Schritt 1 mit den entstandenen Listen

3 1 2 5 9 7 8

Step 4: Wiederholen Schritt 2 mit den entstandenen Listen

1 2 3 5 7 8 9

Step 5: und wieder und wieder und wieder!

1 2 3 5 7 8 9

1 2 3 5 7 8 9

1 2 3 5 7 8 9

Abbildung 1: Ehre wem Ehre gebührt: <https://tex.stackexchange.com/questions/141065/what-are-the-easiest-cleanest-way-to-create-arrays-for-illustrating-quicksort-wi>

- 1.1. (21 Punkte) In dieser Aufgabe sollen Sie eine Version des Quicksort zur lexiographischen Sortierung von positiven Dezimalzahlen implementieren und verschiedene Strategien zur Auswahl des Pivot-Elements ausprobieren. Sie erhalten dazu ein Programmskelett mit bereits definierten Funktionen. Ihre Aufgabe besteht darin, die noch fehlenden Funktionen zu implementieren.

Eine lexiographische Sortierung vergleicht die Zahlen von links nach rechts, wobei die Zahl mit der kleinsten linken Ziffer zuerst kommt. Wenn die linken Ziffern gleich sind, wird die nächste Ziffer verglichen, bis eine Unterscheidung möglich ist. Die Zahlen werden dann entsprechend dieser Reihenfolge sortiert. Beispielsweise ist die lexiographische Sortierung von {3, 30, 34, 300, 345} gleich {3, 30, 300, 34, 345}.

Hinweis:

- Funktionen höherer Ordnung sind in Python möglich und liegen dann vor, wenn eine Funktion eine andere Funktion als Parameter bekommt oder zurückgibt. Beispielsweise ist `def quicksort(arr, pivot_func)` eine solche Funktion. Innerhalb von `def quicksort(arr, pivot_func)` kann dann einfach auf die übergebene Funktion zugegriffen werden, beispielsweise durch `pivot_index = pivot_func(arr)`. Erst beim Aufruf einer Funktion höherer Ordnung muss die übergebene Funktion instanziiert werden, beispielsweise `lexi_sort([], middle_pivot)`.



- In Python können Listen statt mittels `.append()` auch mit arithmetischen Operationen wie `+` oder `+=` verwaltet werden. In dieser Aufgabe dürfen Sie davon Gebrauch machen.

Die zu implementierenden Funktionen sind:

- (a) `def lexi_sort(arr, pivot_func)`: Diese Funktion bekommt eine Liste `arr` bestehend aus dezimalen Ganzzahlen und eine Funktion `pivot_func` als Parameter. Zuerst soll die Liste umgewandelt werden, sodass aus jedem Element eine zweielementige Liste wird, welche aus einer Zahl und einer Liste der Ziffern der Zahl besteht. Beispielsweise soll `[11, 7, 987]` zu `[[11, [1, 1]], [7, [7]], [987, [9, 8, 7]]]` werden. Damit soll anschließend `def quicksort(arr, pivot_func)` aufgerufen werden. Abschließend soll die Umwandlung wieder in einer solchen Weise rückgängig gemacht werden, sodass die zu `arr` lexiographisch sortierte Liste resultiert, welche danach von der Funktion zurückgegeben wird. D.h. aus `[[11, [1, 1]], [7, [7]], [987, [9, 8, 7]]]` wird `[11, 7, 987]` und `[11, 7, 987]` wird zurückgegeben.
- (b) `def quicksort(arr, pivot_func)`: Diese Funktion bekommt eine Liste `arr` und eine Funktion `pivot_func` als Parameter und gibt eine lexiographisch sortierte Liste zurück. Das Ergebnis soll durch Anwenden der Quicksort-Methode mit dem Pivot-Element, welches anhand der übergebenen Funktion `pivot_func` berechnet wird, erstellt werden.
- (c) `def partition(arr, pivot)`: Diese Funktion bekommt eine Liste `arr` und eine Pivot Element `pivot` als Parameter. Durch Anwenden eines lexiographischen Partitionierungsschritts sollen drei Listen erstellt und zurückgegeben werden.
- (d) `def compare_lexicographically(a, b)`: Vergleicht zwei gegebene Listen `a` und `b` von Ziffern einer Dezimalzahl lexiographisch und gibt `True` zurück, falls `a` lexiographisch vor `b` kommt, andernfalls `False`.
- (e) `def num_to_list(x)`: Wandelt eine gegebene ganze positive Dezimalzahl `x` in eine Liste von Ziffern um und gibt diese zurück.
- (f) `def first_pivot(arr)`: Gibt den Index des ersten Elements von `arr` zurück.
- (g) `def last_pivot(arr)`: Gibt den Index des letzten Elements von `arr` zurück.
- (h) `def random_pivot(arr)`: Gibt den Index eines zufälligen Elements von `arr` zurück.
- (i) `def middle_pivot(arr)`: Gibt den Index des mittleren Elements von `arr` zurück.
- (j) `def median_first_middle_last_pivot(arr)`: Gibt den Index des lexiographischen Median-Element des ersten, mittleren und letzten Elements von `arr` als Pivot-Index zurück. Ist dies nicht möglich, wird `def first_pivot(arr)` aufgerufen.
- (k) `def ninther_methode_pivot(arr)`: Gibt den Index des lexiographischen Medians der Dreiermediane aus dem erste, zweite und letzte Drittel des Arrays zurück. Ist dies nicht möglich, wird `def median_first_middle_last_pivot(arr)` aufgerufen.
- (l) `def median_of_three(arr, a, b, c)`: Gibt den Index des lexiographischen Median-Element zu den Indizes `a`, `b` und `c` zurück.
- (m) `def inefficient_pivot(arr)`: Gibt den Index des größten Elements von `arr` zurück.

Mit folgendem Icon können Sie das Programmskelett kopieren. Klicken Sie dafür mit der Linken Maustaste drauf, gegebenenfalls zweimal, oder wählen Sie „Anlage speichern“ nach einem Rechtsklick aus. Verwenden Sie bei Problemen gegebenenfalls ein anderes Programm um die PDF zu öffnen: 

```
import matplotlib.pyplot as plt
import time
import random

def lexi_sort(arr, pivot_func):
    return -1
```



```
def quicksort(arr, pivot_func):
    return -1

def partition(arr, pivot):
    return -1

def compare_lexicographically(a, b):
    return -1

def num_to_list(x):
    return -1

def first_pivot(arr):
    return -1

def last_pivot(arr):
    return -1

def random_pivot(arr):
    return -1

def middle_pivot(arr):
    return -1

def median_first_middle_last_pivot(arr):
    return -1

def ninther_methode_pivot(arr):
    return -1

def median_of_three(arr, a, b, c):
    return -1

def inefficient_pivot(arr):
    return -1

def measure_time(max_n, num_measurements):
    pivot_names = ['Erstes Pivot', 'Letztes Pivot', 'Zufälliges Pivot', 'Mittleres Pivot',
                   'Median Pivot', 'Ninther Pivot', 'Ineffizientes Pivot']
    pivot_colors = ['blue', 'red', 'green', 'cyan', 'yellow', 'magenta', 'black']
    pivot_times = [[0 for j in range(num_measurements)] for i in range(len(pivot_names))]
    iteration_problem_size = max_n // num_measurements
    current_problem_size = iteration_problem_size
    for i in range(num_measurements):
        arr = [random.randint(0, max_n) for i in range(current_problem_size)]
        current_problem_size += iteration_problem_size

    # Hier können Sie Elemente herausnehmen um nur bestimmte Methoden zu plotten
```



```

for k, pivot in enumerate([first_pivot, last_pivot, random_pivot,
                           middle_pivot, median_first_middle_last_pivot,
                           ninther_methode_pivot,
                           inefficient_pivot
                           ]):

    t1 = time.time()
    lexi_sort(arr, pivot)
    t2 = time.time()
    pivot_times[k][i] = t2 - t1
for k, pivot_name in enumerate(pivot_names):
    plt.scatter([], [], color=pivot_colors[k], label=pivot_name, alpha=0.5)
for i in range(num_measurements):
    for k in range(len(pivot_names)):
        plt.scatter(i+1, pivot_times[k][i], color=pivot_colors[k], alpha=0.5)

plt.xlabel('Eingabegröße (n)')
plt.ylabel('Sortierzeit (s)')
plt.legend()
plt.show()

# Beispieltests:
print(lexi_sort([], middle_pivot))
print(lexi_sort([1,2,3,], middle_pivot))
print(lexi_sort([1,2,3,11,73,1,9,38,2,14], middle_pivot))
print(lexi_sort([7,11,987], middle_pivot))
#[ ]
#[1, 2, 3]
#[1, 1, 11, 14, 2, 2, 3, 38, 73, 9]
#[11, 7, 987]

# Zeiten von Sortierung randomisierter Arrays vergleichen:
measure_time(1000, 10)
    
```

Viel Erfolg beim Programmieren!

- 1.2. (12 Punkte) In dieser Aufgabe sollen Sie eine Worst Case Analyse der Laufzeit von Quicksort machen. Im allgemeinen kann die Laufzeit wie folgt beschrieben werden:

$$T_{qs}(n) = T(p) + T(n - p - 1) + \Theta(n)$$

Dabei ist n die Eingabegröße und p die Anzahl an Elementen kleiner (oder größer) als das Pivot Element.

Wie ihnen sicherlich auffällt, unterscheidet sich die Rekurrenzgleichung vom allgemeinen Master Theorem, welches gut für Divide and Conquer Ansätze geeignet ist. Daher ist ihre erste Aufgabe, das Master Theorem für Substract and Conquer herzuleiten. Grundlage bildet folgende Gleichung:

$$T(n) = a \cdot T(n - b) + f(n) \quad \text{mit } b, n > 1 \text{ und } a > 0$$

Leiten Sie damit einem dem Master Theorem ähnliche Fallunterscheidung für $a < 1$, $a = 1$ und $a > 1$ her. Nutzen Sie anschließend ihr Ergebnis um die Worst Case Analyse von Quicksort durchzuführen. Sie dürfen wie auch beim Mastertheorem annehmen, dass $T(n) = \Theta(1)$ gilt, falls n kleiner als ein gewisser Schwellenwert $k > 0$ ist.

2 Hybrider Sortieralgorithmus

2.1. (13 Punkte) In dieser Aufgabe implementieren Sie ein hybrides Sortierverfahren, das die Eigenschaften von Bucketsort mit denen von Heapsort kombiniert. Die zu sortierenden Schlüssel werden nach dem Prinzip von Bucketsort aufgeteilt. Die so vorsortierten Elemente werden dann mit dem Heapsort-Algorithmus endgültig sortiert. Die durchsortierten Buckets abschließend zusammengesetzt und ergeben eine sortierte Liste. Sortiert werden soll eine Liste bestehend aus positiven Ganzzahlen. **Hinweis:** Diese Aufgabe soll Ihnen helfen, die Konzepte hinter nicht vergleichsbasierten Sortieralgorithmen praktisch zu vermitteln. Weitere Informationen zu Bucketsort finden Sie leicht im Internet (z.B. unter https://www.klinger.nrw/wp-content/uploads/2021/01/212_Bucketsort.pdf) oder in unseren Rechnerübungen.

- (a) Implementieren Sie die Python-Funktion `def hybrid_sort(arr, num_buckets)`, die eine Liste `arr` von Zahlen und eine Ganzzahl `num_buckets` als Eingabe erhält. Die Funktion sollte den Bucketsort-Algorithmus mit `num_buckets` Buckets und Mergesort als Unterprozess implementieren, die sortierten Buckets iterativ zusammensetzen und die sortierte Liste zurückgeben.

Um den Bucketsort-Algorithmus zu implementieren, teilen Sie die Eingabeliste in `num_buckets` Buckets auf. Die Bucket-Größe sollte so gewählt werden, dass die Buckets ungefähr die gleiche Größe haben. Dem Index nach sortiert sollen dann die Buckets im Verhältnis von der Anzahl der Buckets zur Länge des Eingabearrays aufsteigend befüllt werden. Beispiel: Bei einer Liste mit 1000 Einträgen und 100 Buckets kommen in den ersten Bucket die Zahlen 1 bis 10, in den zweiten die Zahlen 11 bis 20, etc.

- (b) Implementieren Sie die Python Funktion `def merge_sort(arr)` wie folgt:
1. Wenn die Liste `arr` 0 oder 1 Elemente enthält, ist sie bereits sortiert. Ansonsten teile die Liste in zwei Hälften.
 2. Sortiere beide Hälften rekursiv, indem sie in kleinere Stücke zerteilt und diese zusammengefügt werden.
 3. Füge die sortierten Hälften mittels `def merge(left, right)` zusammen.
- (c) Implementieren Sie die Python Funktion `def merge(left, right)` wie folgt:
1. Vergleiche das erste Element der Listen `left` und `right` miteinander. Füge das kleinere Element in eine Ergebnisliste ein.
 2. Wiederhole Schritt 1, bis alle Elemente aus einer der beiden Listen eingefügt wurden.
 3. Füge die verbleibenden Elemente aus der anderen Liste in die Ergebnisliste ein.

Mit folgendem Icon können Sie das Programmskelett kopieren. Klicken Sie dafür mit der Linken Maustaste drauf, gegebenenfalls zweimal, oder wählen Sie „Anlage speichern“ nach einem Rechtsklick aus. Verwenden Sie bei Problemen gegebenenfalls ein anderes Programm um die PDF zu öffnen: 

```
import random

def hybrid_sort(arr, num_buckets):
    # Bucketsort

    # Rufe Mergesort auf

    # Füge die sortierten Buckets zusammen

    return -1

def merge_sort(arr):

    return -1
```



```
def merge(left, right):  
  
    return -1  
  
# Erstelle eine Liste mit 100 zufälligen Elementen zwischen 1 und 1000  
arr = [random.randint(1, 1000) for _ in range(100)]  
print("Unsortierte Liste: ", arr)  
  
# Sortiere die Liste mittels Bucket Sort mit Mergesort als Unterprozess  
sorted_arr = hybrid_sort(arr, 10)  
print("Sortierte Liste: ", sorted_arr)
```

Viel Erfolg beim Programmieren!

2.2. (4 Punkte) Das hybride Sortierverfahren basiert auf zwei Algorithmen. Im folgenden sind die Laufzeiten und der Speicherbedarf dieser Algorithmen gegeben:

- **Mergesort:**

- Sei n die Eingabegröße.
- Best Case: $\Omega(n \log n)$ Laufzeit und $\Omega(n)$ Speicherbedarf, wenn das Array bereits sortiert ist und keine Rekursion erforderlich ist.
- Worst Case: $\mathcal{O}(n \log n)$ Laufzeit und $\mathcal{O}(n)$ Speicherbedarf, wenn das Array vollständig unsortiert ist und für jede Ebene der Rekursion ein neues Array im Speicher erstellt werden muss.
- Average Case: $\Theta(n \log n)$ Laufzeit und $\Theta(n)$ Speicherbedarf.

- **Bucketsort:**

- Sei k die Anzahl der Buckets.
- Best Case: $\Omega(n + k)$ Laufzeit und $\Omega(n + k)$ Speicherbedarf, wenn die Elemente gleichmäßig auf die Buckets verteilt sind und für jeden Bucket nur eine geringe Anzahl von Elementen vorhanden ist.
- Worst Case: $\mathcal{O}(n^2)$ Laufzeit und $\mathcal{O}(n + k)$ Speicherbedarf, wenn alle Elemente in denselben Bucket fallen und dieser Bucket sortiert werden muss (Bubblesort als Default), z.B. wenn alle Elemente dieselbe Restklasse bei der Modulo-Operation auf die Anzahl der Buckets haben.
- Average Case: $\Theta(n + k)$ Laufzeit und $\Theta(n + k)$ Speicherbedarf, wenn die Elemente zufällig verteilt sind und die Buckets gleichmäßig gefüllt werden.

Leiten Sie damit die Laufzeiten und den benötigten Speicherplatz für den Best Case, Average Case und Worst Case ab.